

Day 8: STTP on Ethical Hacking and Cyber Forensics @IIITK

Disk Forensics using Autopsy & Reverse Engineering Mobile Apps

Dr. Sreelakshmy I.J & Dr.Abin Oommen Philip

June 18, 2025

Contents

1	Disk Analysis Using Autopsy	2
1.1	Introduction	2
1.1.1	Hacking Case Scenario	2
1.2	Setting Up Autopsy	2
1.2.1	Installation	2
1.2.2	Creating a New Case	2
1.2.3	Adding a Data Source	3
1.3	Analyzing Data in Autopsy	3
1.4	Answers to Basic Case Questions	4
1.4.1	Additional Basic Findings	6
1.5	Generating a Report	9
2	Reverse Engineering Mobile Apps	10
2.1	Introduction	10
2.2	Tools Required	10
2.3	Using Apktool	10
2.3.1	Installation of Apktool	10
2.3.2	Disassembling an APK	11
2.3.3	Locating and Modifying Smali Code	11
2.3.4	Reassembling the APK	12
2.3.5	Signing the APK	12
2.4	Using Jadx-gui	13
2.5	Basic Analysis of a Suspect APK	13
2.6	Mobile Forensics Basics (Android)	14
2.6.1	Data Extraction from Android	14

1 Disk Analysis Using Autopsy

1.1 Introduction

Autopsy is a digital forensics platform used to investigate disk images and analyze file systems.

1.1.1 Hacking Case Scenario

The analysis is based on a scenario involving a Dell CPI notebook suspected of being used for hacking by "Greg Schardt" aka "Mr. Evil". The goal is to find hacking software, evidence of use, generated data, and tie the computer to the suspect. A forensic image has been made.

1.2 Setting Up Autopsy

1.2.1 Installation

Download and install Autopsy on your analysis machine.

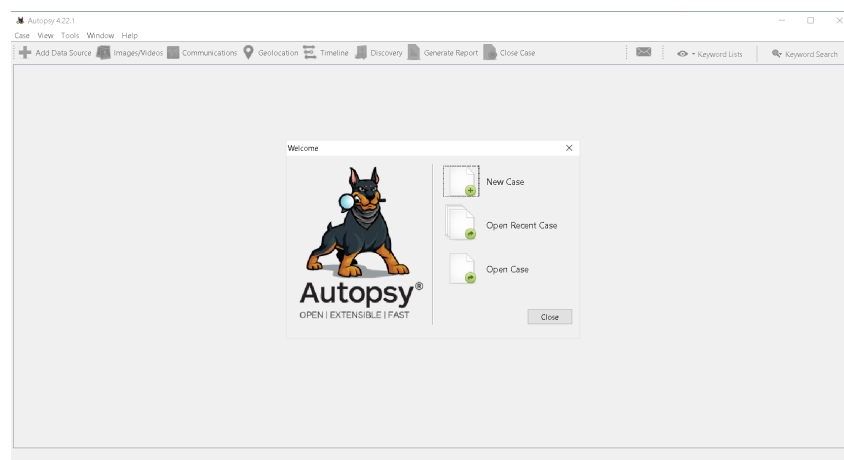


Figure 1: Autopsy Welcome Screen

1.2.2 Creating a New Case

1. Launch Autopsy and select "New Case".
2. **Case Information:** Enter a Case Name and specify the Base Directory for storing case data. Choose the Case Type.

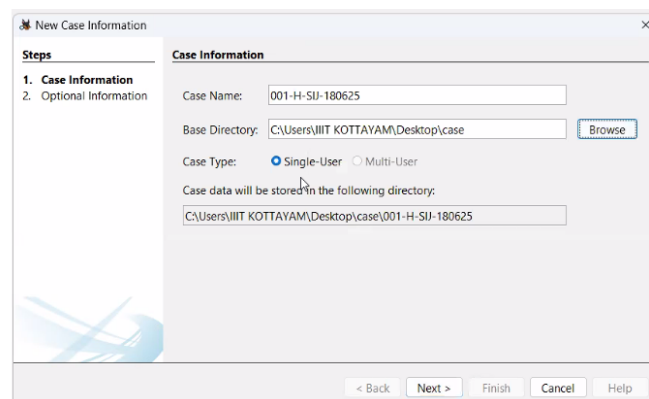


Figure 2: New Case Information

3. **Optional Information:** Enter examiner details. Click "Finish".

1.2.3 Adding a Data Source

1. **Select Host:** Choose to generate a new host name or specify one.
2. **Select Data Source Type:** Choose "Disk Image or VM File".

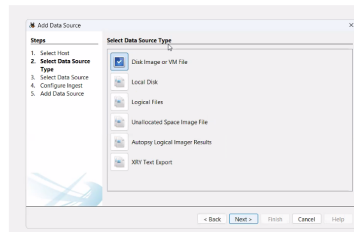


Figure 3: Select Data Source Type

3. **Select Data Source:** Browse and select the disk image file. Set the appropriate Time zone.

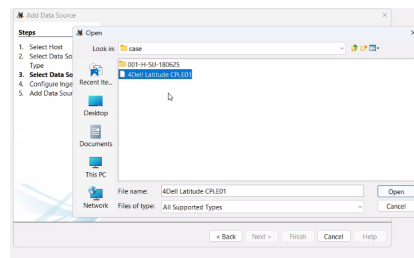


Figure 4: Select Data Source Path

4. **Configure Ingest Modules:** Select analysis modules like Recent Activity, Hash Lookup, File Type Identification, and Keyword Search.
5. **Add Data Source:** Autopsy processes the data source. This may take time.

1.3 Analyzing Data in Autopsy

Explore data using the tree view:

- **Data Sources:** Disk images and their structure.
- **File Views:** Files by type, deleted files, etc.
- **Data Artifacts:** Extracted items like Installed Programs, OS Information, Recent Documents, USB Devices.
- **Analysis Results:** Module outputs like Extension Mismatches, Keyword Hits.

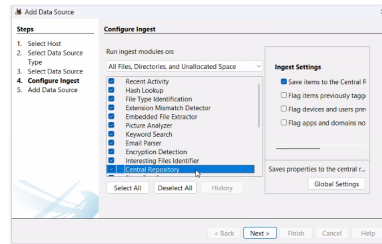


Figure 5: Configure Ingest Modules

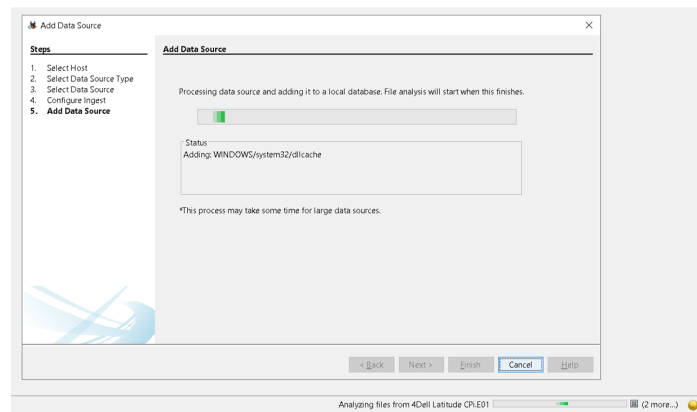


Figure 6: Adding Data Source Progress

- **OS Accounts:** User accounts.

1.4 Answers to Basic Case Questions

1. **Image hash (MD5):** aee4fcd9301c03b3b054623ca261959a.
2. **Operating system:** Microsoft Windows XP.
3. **Registered owner:** Greg Schardt.
4. **Install date:** Accounts like Administrator were created on **2004-08-19**. The image acquisition was on **2004-09-22**.
5. **Computer account name:** N-1A90DN6ZXK4LQ.
6. **User accounts recorded:** 5 main user accounts including Administrator, Guest, and Mr. Evil.
7. **Primary user account name:** Mr. Evil, based on login counts and home directory usage.
8. **Last user to logon:** mr. evil, on 2004-08-27.
9. **3 installed programs possibly used for hacking:** Ethereal, Network Stumbler, Cain & Abel.

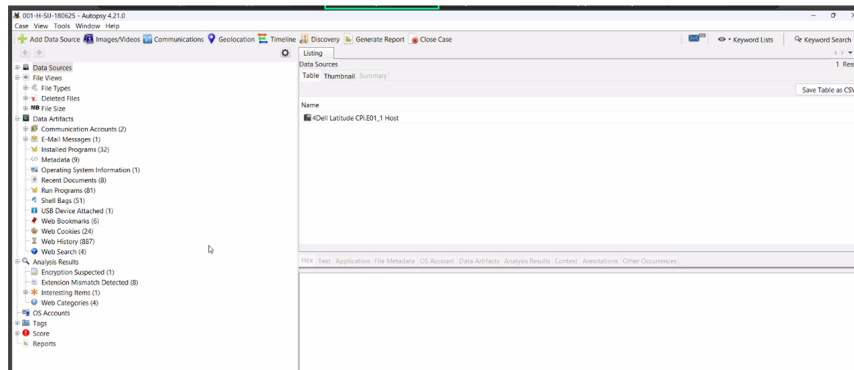


Figure 7: Autopsy Main Interface

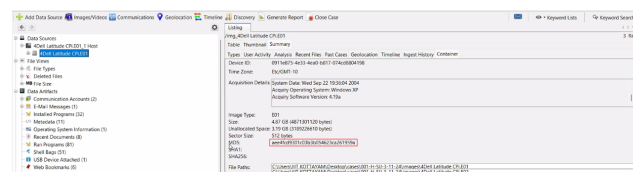


Figure 8: Image Hash Value

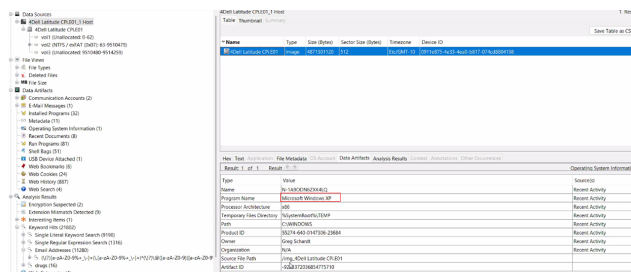


Figure 9: Operating System Information

Table: **Thumbnail Summary**

Name	S	C	O	Login Name	Host	Scope	Profile Name	Creation Time
5-5-19-2000070504-68070804-11000	0	0	0	5-5-19	4Cell Latitude CP1E01	Local	NT AUTHORITY	2004-08-20 09:15:10 IST
5-5-19-2000070504-68070804-11000	0	0	0	5-5-19	4Cell Latitude CP1E01	Local	NT AUTHORITY	2004-08-20 09:15:10 IST
5-5-19-2000070504-68070804-11000	0	0	0	5-5-19	4Cell Latitude CP1E01	Local	NT AUTHORITY	2004-08-20 09:15:10 IST
5-5-19-2000070504-68070804-11000	0	0	0	5-5-19	4Cell Latitude CP1E01	Local	NT AUTHORITY	2004-08-20 09:15:10 IST
5-5-19-2000070504-68070804-11000	0	0	0	5-5-19	4Cell Latitude CP1E01	Local	NT AUTHORITY	2004-08-20 09:15:10 IST

Figure 13: List of OS Accounts

Table: **Thumbnail Summary**

Name	S	C	O	Login Name	Host	Scope	Profile Name	Creation Time
5-5-19-2000070504-68070804-11000	0	0	0	5-5-19	4Cell Latitude CP1E01	Local	NT AUTHORITY	2004-08-20 09:15:10 IST
5-5-19-2000070504-68070804-11000	0	0	0	5-5-19	4Cell Latitude CP1E01	Local	NT AUTHORITY	2004-08-20 09:15:10 IST
5-5-19-2000070504-68070804-11000	0	0	0	5-5-19	4Cell Latitude CP1E01	Local	NT AUTHORITY	2004-08-20 09:15:10 IST
5-5-19-2000070504-68070804-11000	0	0	0	5-5-19	4Cell Latitude CP1E01	Local	NT AUTHORITY	2004-08-20 09:15:10 IST
5-5-19-2000070504-68070804-11000	0	0	0	5-5-19	4Cell Latitude CP1E01	Local	NT AUTHORITY	2004-08-20 09:15:10 IST

Figure 14: Mr. Evil Login Details

Table: **Thumbnail Summary**

Source Name	S	C	O	Program Name	Date/Time	Data Source
software	0	0	0	Ethernet 0.10.6 v.0.10.6	2004-08-27 15:20:19 IST	4Cell Latitude CP1E01
software	0	0	0	WinPcap 3.01 alpha	2004-08-27 15:15:19 IST	4Cell Latitude CP1E01
software	0	0	0	Network Sniffer 0.8.0 (remove only)	2004-08-27 15:12:15 IST	4Cell Latitude CP1E01
software	0	0	0	Look@LAN 2.50 Build 29	2004-08-25 15:56:11 IST	4Cell Latitude CP1E01
software	0	0	0	123 Write All Stored Passwords	2004-08-20 15:13:08 IST	4Cell Latitude CP1E01
software	0	0	0	PowerToys for Windows XP v.1.0.0.0000	2004-08-20 15:12:43 IST	4Cell Latitude CP1E01
software	0	0	0	mIRC	2004-08-20 15:10:04 IST	4Cell Latitude CP1E01
software	0	0	0	CamTill	2004-08-20 15:09:03 IST	4Cell Latitude CP1E01
software	0	0	0	CodeTill	2004-08-20 15:09:02 IST	4Cell Latitude CP1E01
software	0	0	0	Forti Agent	2004-08-20 15:08:19 IST	4Cell Latitude CP1E01
software	0	0	0	Fiber Toys v.2.4 Build 216	2004-08-20 15:07:25 IST	4Cell Latitude CP1E01
software	0	0	0	Cam & Abel v2.5 beta45	2004-08-20 15:05:58 IST	4Cell Latitude CP1E01
software	0	0	0	Anonymizer Bar 2.0 (remove only)	2004-08-20 15:05:09 IST	4Cell Latitude CP1E01

Figure 15: Installed Programs (Potential Hacking Tools)

12. The IP address 192.168.1.111 and MAC 0010a4933e09 are found in `irunin.ini`.

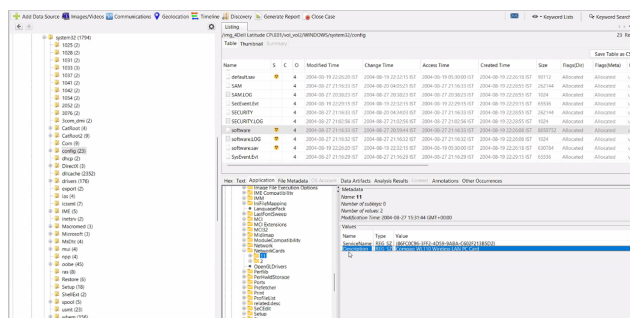


Figure 16: Network Card Information

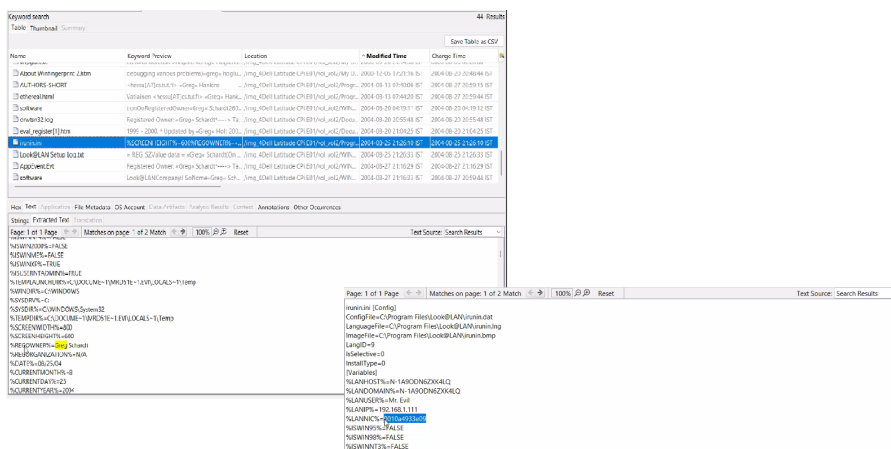


Figure 17: irunin.ini content

13. A file named `interception` exists in Mr. Evil's My Documents folder, likely containing sniffed data.

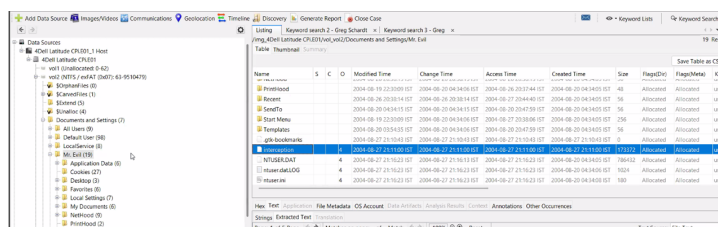


Figure 18: ‘interception’ File

14. Four executable files (Dc1.exe, Dc2.exe, Dc3.exe, Dc4.exe) are in the Recycle Bin.

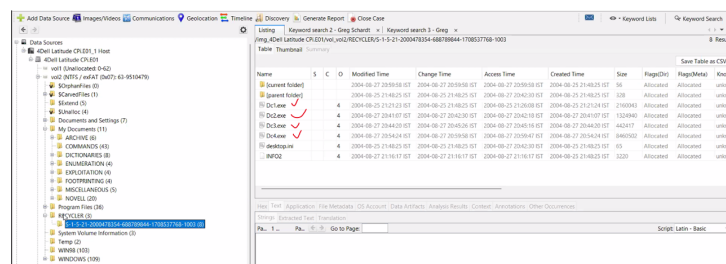


Figure 19: Executables in Recycle Bin

1.5 Generating a Report

Autopsy can generate reports (e.g., HTML) of findings.

1. Click "Generate Report". Select data sources and results to include.

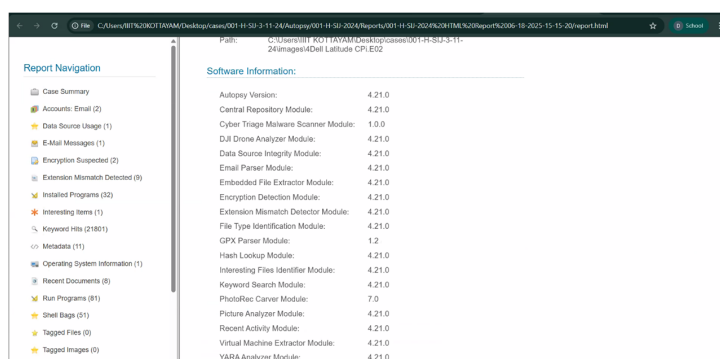


Figure 20: Autopsy HTML Report Sample

2 Reverse Engineering Mobile Apps

2.1 Introduction

Reverse engineering mobile APKs involves decompiling and analyzing an application's code and resources to understand its functionality or identify vulnerabilities. Genymotion emulator is often used for testing modified apps.

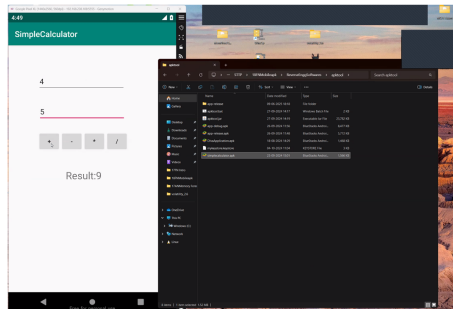


Figure 21: Tools for Reverse Engineering

2.2 Tools Required

1. **Apktool:** For disassembling and reassembling APK files.
2. **Jadx-gui:** For decompiling APKs to Java code for analysis.

2.3 Using Apktool

2.3.1 Installation of Apktool

1. Ensure Java (JDK) is installed.
2. Download the Windows wrapper script (`apktool.bat`) and the latest Apktool JAR (`apktool.jar`).
3. Place both files in your Windows directory or another directory in your system's PATH.
4. Test by running `apktool` in the command prompt.

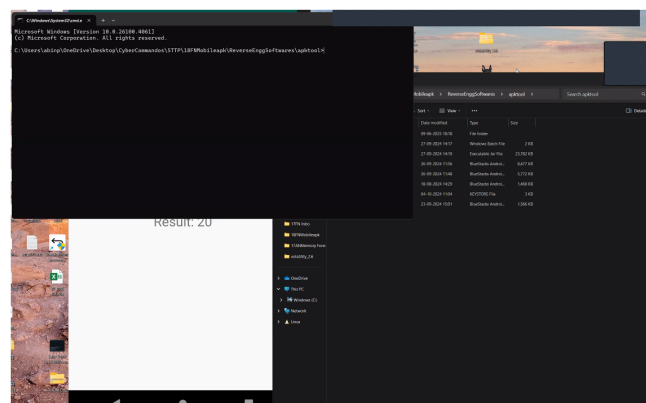


Figure 22: Apktool in Command Prompt

2.3.2 Disassembling an APK

1. Command: `apktool d filename.apk`

```
C:\...\apktool> apktool d simplecalculator.apk
```

2. This creates a folder with disassembled contents (Smali files, resources).

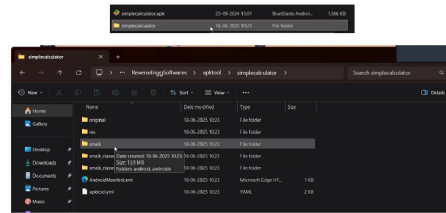


Figure 23: Disassembled APK Folder Structure

2.3.3 Locating and Modifying Smali Code

Smali is an assembly language for Dalvik. Modifications are made to Smali files.

- Smali files are found in folders like `smali`, `smali_classes...`
- Example modification: Changing `add-int` to `mul-int` in a `.smali` file.

```
line 29  
local v1, "number2"  
add-int v2, v0, v1  
  
line 30  
local v2, "result"  
get-object v3, p0, Lcom/stillamencioglu/simplecalculator/MainActivity->  
mTextView.findViewById<TextView>
```

```
line 29  
local v1, "number2"  
mul-int v2, v0, v1  
  
line 30  
local v2, "result"  
get-object v3, p0, Lcom/stillamencioglu/simplecalculator/MainActivity->  
mTextView.findViewById<TextView>
```

Figure 24: Modifying Smali Code

Note: Java code from decompilers like Jadx cannot be directly recompiled into an APK using Apktool. Modifications are done at the Smali level.

2.3.4 Reassembling the APK

1. Command: `apktool b foldername`
2. The reassembled APK will be in the `dist` subfolder.

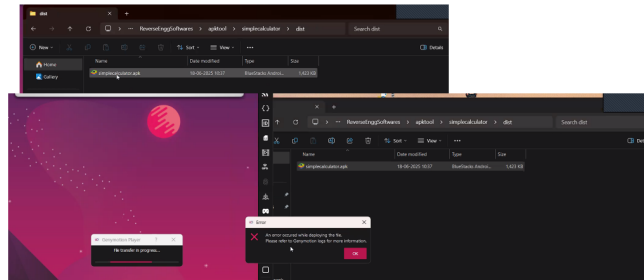


Figure 25: Reassembling APK

2.3.5 Signing the APK

Reassembled APKs must be signed to be installed.

1. **Zipalign** (aligns data): `zipalign -v 4 infile.apk outfile_aligned.apk`
2. **Generate a Keystore** (if needed):

```
keytool -genkey -v -keystore mykeystore.keystore -alias alias_name -keyalg RSA -  
keysize 2048 -validity 10000
```

```
C:\Users\abinp\OneDrive\Desktop\CyberCommands\STTP\18FNMobileapk\ReverseEnggSoftwares\apktool\simplecalculator\dist>keytool -v -keystore mykeystore.keystore -alias -alias_name -keyalg RSA -keysize 2048 -validity 10000  
Enter keystore password:  
Re-enter new password:  
Enter the distinguished name. Provide a single dot (.) to leave a sub-component empty or press ENTER to use the default  
ces.  
What is your first and last name?  
What is the name of your organizational unit?  
What is the name of your organization?  
What is the name of your City or Locality?  
What is the name of your State or Province?  
What is the two-letter country code for this unit?  
Is C=Unknown, OU=Unknown, O=Unknown, L=Unknown, ST=Unknown, C=Unknown correct?  
[no]: y  
Generating 2,048 bit RSA key pair and self-signed certificate (SHA384withRSA) with a validity of 10,000 days  
for: C=Unknown, OU=Unknown, O=Unknown, L=Unknown, ST=Unknown, C=Unknown  
[Storing mykeystore.keystore]  
C:\Users\abinp\OneDrive\Desktop\CyberCommands\STTP\18FNMobileapk\ReverseEnggSoftwares\apktool\simplecalculator\dist>
```

Figure 26: Keystore Generation

3. Sign the APK with `apksigner`:

```
apksigner sign --ks mykeystore.keystore aligned_apk_name.apk
```

```
C:\Users\abinp\OneDrive\Desktop\CyberCommander\STTP\18F\NMobileapk\ReverseEnggSoftwares\apktool\simplecalculator\dist>apksigner sign --ks mykeystore.keystore --v1-signing-enabled true --v2-signing-enabled true C:\Users\abinp\OneDrive\Desktop\CyberCommander\STTP\18F\NMobileapk\ReverseEnggSoftwares\apktool\simplecalculator\dist\zipaligncalc.apk  
Keystore password for signer #1:
```

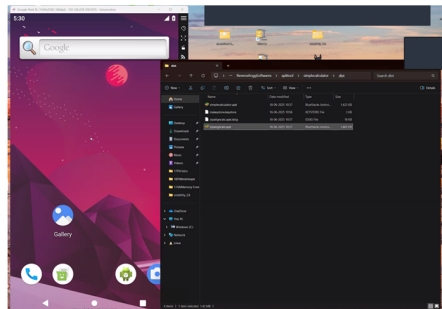


Figure 27: Signing APK

4. **Verify Signature** (optional): `apksigner verify signed_apk_name.apk`

The modified and signed app can then be tested.



Figure 28: Result of Modified App

2.4 Using Jadx-gui

Jadx-gui decompiles APKs to Java code for easier understanding.

1. Download and extract Jadx and Jadx-gui.
2. **Jadx-gui:** Open `jadx-gui.exe`. Open an APK file or a project folder decompiled by command-line jadx.

2.5 Basic Analysis of a Suspect APK

1. **Decompile using Jadx-gui:** Open the suspect APK.
2. **Manifest File Analysis** (`AndroidManifest.xml`):
 - Check for requested permissions. Suspicious permissions for a banking app might include `RECEIVE_SMS`, `SEND_SMS`.
3. **Source Code Analysis:**
 - Look for code obfuscation (mangled names).

```

import android.view.View;
import android.widget.EditText;
import android.widget.TextView;
import androidx.appcompat.app.AppCompatActivity;

/* Loaded from: classes.dex */
public class MainActivity extends AppCompatActivity {

    /* JADX INFO: Access modifiers changed from: protected */
    @Override // androidx.appcompat.app.AppCompatActivity, androidx.fragment.app.FragmentActivity, androidx.core.
    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        this.number1Text = (EditText) findViewById(R.id.number1Text);
        this.number2Text = (EditText) findViewById(R.id.number2Text);
        this.resultText = (TextView) findViewById(R.id.resultText);
    }

    public void sum(View view) {
        if (this.number1Text.getText().toString().matches("") || this.number2Text.getText().toString().matches(""))
            return;
        this.resultText.setText("Enter Number!");
    }

    public void deduct(View view) {
        if (this.number1Text.getText().toString().matches("") || this.number2Text.getText().toString().matches(""))
            return;
        this.resultText.setText("Enter Number!");
    }
}

```

Figure 29: Viewing Decompiled Code in Jadx-gui

```

<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    android:versionCode="1"
    android:versionName="1.0"
    android:compileSdkVersion="33"
    android:compileSdkVersionCodename="13"
    package="com.example.myapplication"
    platformBuildVersionName="13">
    <uses-sdk
        android:minSdkVersion="24"
        android:targetSdkVersion="33"/>
    <uses-permission android:name="android.permission.INTERNET"/>
    <uses-permission android:name="android.permission.RECEIVE_SMS"/>
    <uses-permission android:name="android.permission.SEND_SMS"/>
    <activity
        android:name="com.example.myapplication.MainActivity"
        android:exported="true"
        <intent-filter>
            <action android:name="android.intent.action.MAIN"/>
            <category android:name="android.intent.category.LAUNCHER"/>
        </intent-filter>
    </activity>
</manifest>

```

Figure 30: Checking AndroidManifest.xml for Permissions

- Identify collection of sensitive data (PINs, account numbers).
 - Check for data being sent to hardcoded phone numbers or URLs.
4. Basic findings might reveal data theft mechanisms or phishing behavior.

2.6 Mobile Forensics Basics (Android)

2.6.1 Data Extraction from Android

1. **Prerequisites:** ADB (Android Debug Bridge) on PC, USB Debugging enabled on device.
2. **ADB Commands:**

```

adb devices # List connected devices
adb shell # Open a shell on the device

```

3. **Root Access:** May be needed for full data access (su command).

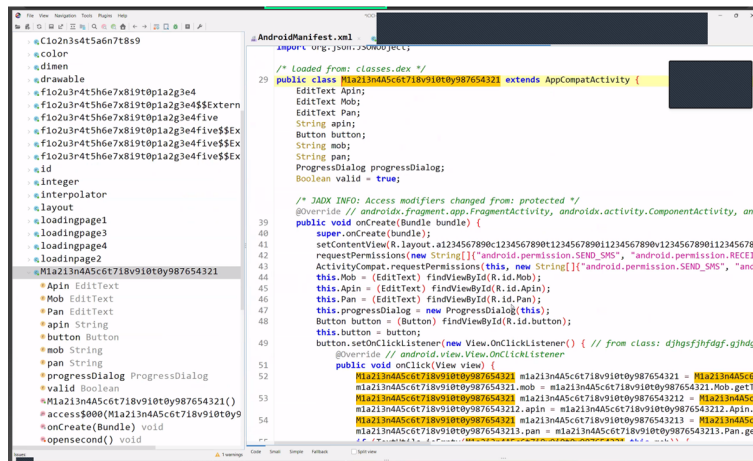


Figure 31: Example of Obfuscated Code

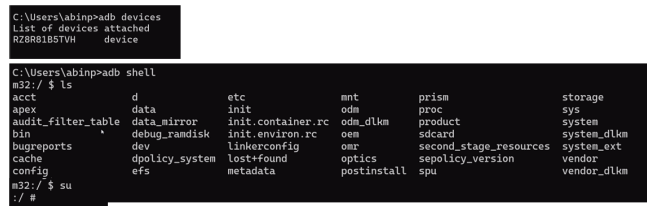


Figure 32: Basic ADB Commands

4. **Navigate File System:** Use `cd` and `ls` to explore app data folders like `/data/data/<app.package.name>`.
 5. **Key Data Locations:**
 - **shared_prefs:** Stores app preferences (XML files).
 - **databases:** SQLite databases storing app data.
 6. **Analyzing Databases:** Pull database files using `adb pull` and view with a SQLite browser.
- Specialized forensic tools like Magnet AXIOM can also be used for mobile data acquisition.

```
com.facebook.katana
com.facebook.services
com.facebook.system
com.fitbit.FitbitMobile
com.gabitrn

./data/data # cd com.fitbit.FitbitMobile
./data/data/com.fitbit.FitbitMobile # ls
app_companions      app_textures      cache              files
app_fitbit-packages-tmp  app_webview      code_cache         no_backup
app_i2wfiles         app_webview_com.fitbit.FitbitMobile  databases          shared_prefs
```

Figure 33: App Data Folder

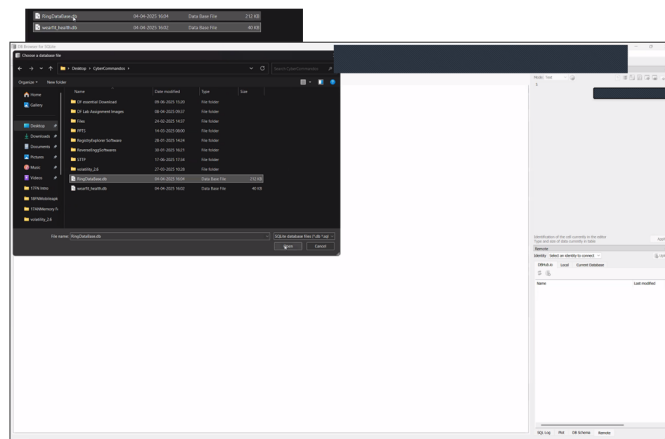


Figure 34: DB Browser for SQLite

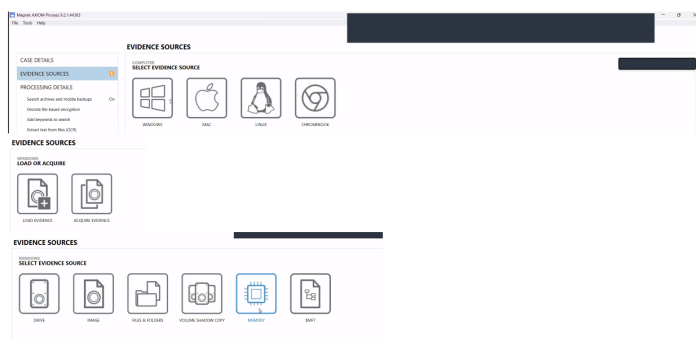


Figure 35: Magnet AXIOM Evidence Selection